

See discussions, stats, and author profiles for this publication at: <https://www.researchgate.net/publication/313037581>

Reconciliation of RDF* and Property Graphs

Technical Report · November 2014

CITATIONS

23

READS

352

1 author:



[Olaf Hartig](#)

Linköping University

90 PUBLICATIONS 1,719 CITATIONS

SEE PROFILE

Some of the authors of this publication are also working on these related projects:



Linked Data Fragments [View project](#)

Reconciliation of RDF^{*} and Property Graphs

Olaf Hartig

University of Waterloo

<http://olafhartig.de>

November 14, 2014

Abstract

Both the notion of Property Graphs (PG) and the Resource Description Framework (RDF) are commonly used models for representing graph-shaped data. While there exist some system-specific solutions to convert data from one model to the other, these solutions are not entirely compatible with one another and none of them appears to be based on a formal foundation. In fact, for the PG model, there does not even exist a commonly agreed-upon formal definition.

The aim of this document is to reconcile both models formally. To this end, the document proposes a formalization of the PG model and introduces well-defined transformations between PGs and RDF. As a result, the document provides a basis for the following two innovations: On one hand, by implementing the RDF-to-PG transformations defined in this document, PG-based systems can enable their users to load RDF data and make it accessible in a *compatible*, system-independent manner using, e.g., the graph traversal language Gremlin or the declarative graph query language Cypher. On the other hand, the PG-to-RDF transformation in this document enables RDF data management systems to support *compatible*, system-independent queries over the content of Property Graphs by using the standard RDF query language SPARQL. Additionally, this document represents a foundation for systematic research on relationships between the two models and between their query languages.

If you have comments or suggestions about this proposal, as well as implementations or applications thereof, do not hesitate to let the author know about them.

1 Introduction

This document reconciles two commonly used graph-based data models, namely the Property Graphs model which is used by popular graph database systems such as Neo4j¹, Titan² and Sparksee³, and the RDF data model [CWL⁺14], which has been standardized by the World Wide Web Consortium (W3C) and is supported by numerous systems including IBM's DB2⁴, OpenLink's Virtuoso⁵ and Systap's Bigdata⁶. The primary goal of this reconciliation is to enable a user who is familiar with either of these data models to access data, represented in the other model, based on a *well-defined, system-independent view* of this data given in the model most familiar to the user.

¹<http://neo4j.org/>

²<http://thinkarelius.github.com/titan/>

³<http://sparsity-technologies.com/#sparksee>

⁴<http://www-01.ibm.com/software/data/db2/linux-unix-windows/nosql-support.html>

⁵<http://virtuoso.openlinksw.com/>

⁶<http://www.systap.com/>

The document is structured as follows: Section 2 gives a very brief informal overview of Property Graphs, RDF, and an extension of RDF that provides for a more user-friendly representation of statement-level metadata and serves as a basis of the reconciliation. Section 3 then describes the proposal informally. Thereafter, Sections 4 to 7 provide the formal details.

2 Informal Overview of the Data Models

The notion of a Property Graph has been described informally in a number of books, tutorials, manuals, and other documentation on the Web. Perhaps one of the most representative descriptions is the following by Robinson et al. [RWE13].

- “A property graph is made up of nodes, relationships, and properties.
- Nodes contain properties [...] in the form of arbitrary key-value pairs. The keys are strings and the values are arbitrary data types.
- [...] A relationship always has a direction, a label, and a start node and an end node [...].
- Like nodes, relationships can also have properties.” [RWE13]

Example 1. Figure 1 illustrates a simple Property Graph with two nodes and two relationships between them. One of these relationships has the label mentioned, the start node is the node for Orson Welles, and the end node is the node for Stanley Kubrick. The other relationship, labeled influencedBy, starts from the Kubrick node and ends in the Welles node. The light gray boxes associated with some of the graph elements represent properties of these elements. For instance, the node for Stanley Kubrick has two properties giving the name and the birth year of the famous director, and the influencedBy relationship has a property that specifies the certainty of whether Kubrick was influenced by Welles. The other relationship does not have a property in the given graph.

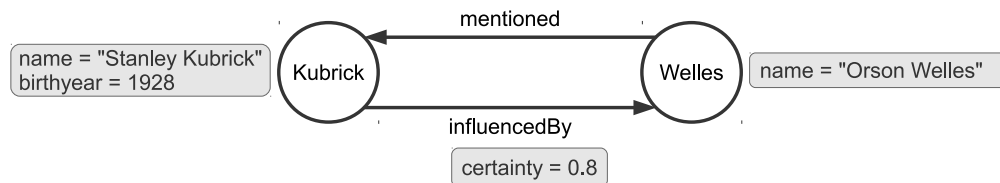


Figure 1: A Property Graph with two nodes.

In contrast to Property Graphs, RDF is a standardized data model. This model represents data as sets of triples where each triple consists of three elements that are referred to as the subject, the predicate, and the object of the triple. These triples allow users to describe arbitrary things in terms of their attributes and their relationships to other things. That is, such things may be the subject or object of an RDF triple in which, usually, they are denoted by an Internationalized Resource Identifier (IRI)—which is a form of a Web-wide unique identifier (hence, in contrast to vertex and edge identity that exists only within a given Property Graph, IRIs identify things across datasets). Relationships and attributes are also denoted by IRIs, which appear as the predicate of RDF triples. Finally, attribute values in RDF are called literals and appear in the object of RDF triples. Any set of RDF triples can be conceived of as a directed graph in which each triple

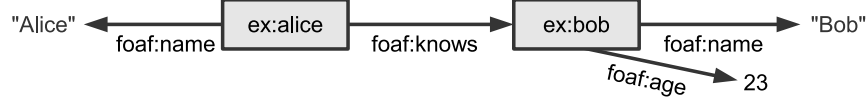


Figure 2: A simple RDF graph consisting of four RDF triples.

represents an edge from the subject to the object of that triple; hence, the vertices in this graph are all subjects and objects of all triples in the set.

Example 2. *The following RDF data, represented in the Turtle format [PC14], contains four RDF triples that describe two persons, Alice and Bob, denoted by the IRIs `http://example.org/alice` and `http://example.org/bob`, respectively. The objects of the last three triples are literals. Figure 2 illustrates the graph representation of these four triples.*

```
@prefix foaf: <http://xmlns.com/foaf/0.1/> .
@prefix ex: <http://example.org/> .

ex:alice foaf:knows ex:bob .
ex:alice foaf:name "Alice" .
ex:bob foaf:name "Bob" .
ex:bob foaf:age 23 .
```

A shortcoming that RDF has been widely criticized for is the lack of an approach to represent statement-level metadata that is as intuitive and user-friendly as edge properties in Property Graphs (such as the certainty statement in the sample Property Graph in Figure 1). While RDF provides a notion of reification to support this use case [HPS14], this approach is awkward to use, the resulting metadata is cumbersome to query, and it may blow up the dataset size significantly. However, a recently proposed extension of RDF addresses this shortcoming by making *triples about triples* a first class citizen in the data model [HT14]. That is, this extension, which is called RDF*, augments RDF with the possibility to use a triple directly as the subject or object of other triples.

Example 3. *Consider an extended version of the RDF Turtle format, called Turtle* [HT14], that implements the idea of embedding RDF triples into other RDF triples by enclosing the embedded triple in '<<' and '>>'. Then, the data in Example 2 can be augmented with metadata as follows:*

```
<<ex:alice foaf:knows ex:bob>> ex:certainty 0.5 .
ex:alice foaf:name "Alice" .
ex:bob foaf:name "Bob" .
<<ex:bob foaf:age 23>> ex:certainty 0.9 .
```

Note that the last line represents a metadata triple whose subject is the triple that provides information about the age of Bob, and the metadata triple describes the certainty of this information. Similarly, the first line represents a metadata triple about the triple that is embedded as subject.

It is important to emphasize that RDF* is simply a syntactic extension of RDF that makes dealing with statement-level metadata more intuitive. In fact, there exists a well-defined transformation⁷ of RDF* data back to standard RDF data [HT14]. On the other hand, standard RDF data can be understood trivially as RDF* data. This document leverages these properties and uses RDF* as a basis for reconciling the RDF data model and the Property Graphs model.

⁷In a nutshell, this transformation employs standard RDF reification to convert any RDF* metadata triple into a set of ordinary RDF triples [HT14].

3 Informal Overview of the Proposal

This document *formalizes* three transformations: two from RDF* data to Property Graphs, and one from Property Graphs to RDF* data. All three transformations also cover ordinary RDF data because of the aforementioned relationship between RDF* and the standard RDF data model. This section provides an informal overview of the transformations and outlines some use cases.

3.1 First Transformation: RDF* to RDF-like Property Graphs

The first transformation presents an intuitive (perhaps the most natural) way of converting RDF* data to Property Graphs; namely, this transformation represents any ordinary RDF triple as an edge in the resulting Property Graph; the two vertices incident to such an edge have properties that describe the subject and the object of the corresponding RDF triple; and metadata triples are represented as edge properties.

Example 4. *Figure 3 illustrates the Property Graph that can be obtained by transforming the RDF* data in Example 3 based on the proposed transformation.*

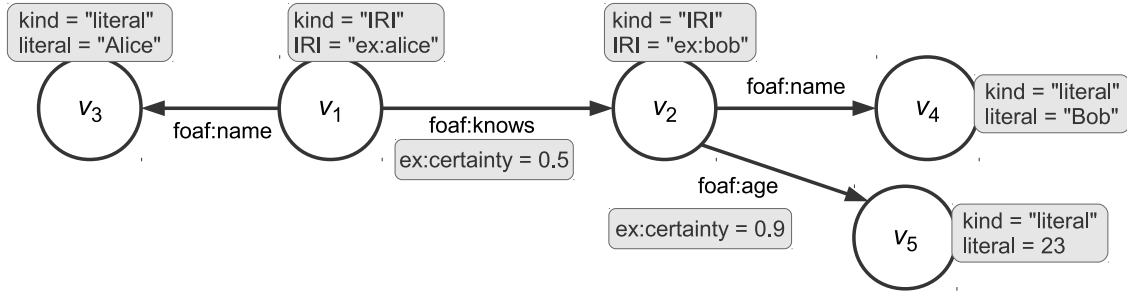


Figure 3: An (RDF-like) Property Graph that represents the RDF* data in Example 3.

A caveat of the straightforward approach to represent RDF* data as Property Graphs is that the transformation is not applicable in some cases because of differences in the expressiveness of RDF* and RDF on the one hand and Property Graphs on the other. For instance, the object of a metadata triple (with an RDF triple as its subject) may be an IRI that is also mentioned in other triples, whereas the value of an edge (or vertex) property cannot be a vertex itself. For a more detailed discussion of these differences and a formal characterization of the RDF* data that the transformation can be applied to, refer to Section 4.3. A formalization of the transformation itself can be found in Section 5.

Even if not generally applicable, for the large number of cases in which the transformation can be applied, the transformation can be used to enable users who are familiar with the graph query language Cypher to query RDF/RDF* data without having to learn the RDF query language SPARQL.

Example 5. *The following Cypher query can be used to query the sample data to find the name of persons known by somebody whose name is Alice (an execution over the sample data would return a single value, namely, Bob).*

```
START a=node(*)
MATCH (a)-[:foaf:name]->(bn { literal="Alice" }),
      (a)-[:foaf:knows]->(p)-[:foaf:name]->(pn)
RETURN pn.literal
```

An equivalent⁸ SPARQL representation of this Cypher query is given as follows.

```
SELECT ?pn WHERE {
  ?a foaf:name "Alice" .
  ?a foaf:knows ?p .
  ?p foaf:name ?pn }
```

The example demonstrates that Property Graphs obtained by the proposed transformation can be queried using Cypher queries for which there exist corresponding SPARQL over the original RDF (or RDF^{*}) data. However, the transformation also enables users to benefit from features of Cypher that are not available in SPARQL. For instance, Cypher allows for path expressions that are more powerful than the property path feature provided by SPARQL 1.1. Another example is querying statement-level metadata by accessing the corresponding edge properties using Cypher.

Example 6. Consider an extension of the query in the previous example with the additional condition that the certainty of the information that Alice knows the listed persons must be greater than 0.7. The following Cypher query captures this condition by filtering on the corresponding edge property.

```
START a=node(*)
MATCH (a)-[:foaf:name]->(bn { literal="Alice" }),
      (a)-[r:foaf:knows]->(p)-[:foaf:name]->(pn)
WHERE r.ex:certainty > 0.7
RETURN pn.literal
```

Any system that is based on Property Graphs can enable users to query RDF (or RDF^{*}) data using Cypher as outlined in the examples. To this end, the system only has to implement an import procedure that applies the first (or the second) transformation as formalized in this document. On the other hand, any RDF-based system—independent of whether it supports the RDF^{*} extension or not—may support Cypher queries on top of a virtual Property Graph view of the data (where the view is defined by the given transformation). However, for RDF^{*}-enabled systems, if the primary use case for supporting Cypher are more user-friendly queries over statement-level metadata (as demonstrated in Example 6), a more native approach is SPARQL^{*}, which is an RDF^{*}-specific extension of SPARQL that carries over the idea of embedding triples and comes with a well-defined query semantics [HT14].

Example 7. The following SPARQL^{*} query is equivalent to the Cypher query in Example 6.

```
SELECT ?pn WHERE {
  ?a foaf:name "Alice" .
  <<?a foaf:knows ?p>> ex:certainty ?c .
  FILTER ( ?c > 0.7 )
  ?p foaf:name ?pn }
```

An alternative form of this SPARQL^{*} query is to use a BIND clause as follows.

```
SELECT ?pn WHERE {
  ?a foaf:name "Alice" .
  BIND( <<?a foaf:knows ?p>> AS ?t )
  ?t ex:certainty ?c .
  FILTER ( ?c > 0.7 )
  ?p foaf:name ?pn }
```

⁸The equivalence cannot be shown formally because Cypher does not have a formally defined query semantics.

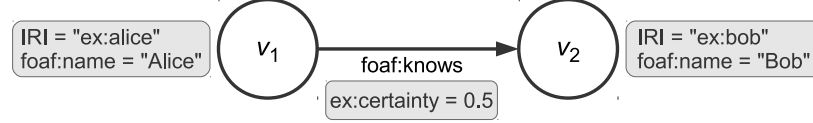


Figure 4: A simple Property Graph that represents the RDF* data in Example 3 partially.

3.2 Second Transformation: RDF* to Simple Property Graphs

Property Graphs as obtained by the transformation described in the previous section resemble the structure of RDF/RDF* data. In fact, any of these Property Graphs contains all information present in the original RDF* data. Hence, the transformation is lossless. However, some users of Property Graph systems may consider the resulting Property Graphs unnatural or too complex. Therefore, this document introduces another transformation for converting RDF* data to Property Graphs that have a simpler structure, but cannot be used to reconstruct the original RDF* data.

The transformation distinguishes *attribute triples*, that is, ordinary (non-metadata) triples whose object is a literal, and *relationship triples*, that is, ordinary triples whose object is an IRI or a blank node. Then, the transformation represents every relationship triple as an edge; every attribute triple is converted into a property of the vertex for the subject of that triple (instead of also converting it into a separate edge as the lossless transformation does). Hence, vertices in the resulting Property Graph represent IRIs and blank nodes only (whereas the lossless transformation produces vertices that may represent literals). Metadata triples about relationship triples are converted into edge properties, whereas metadata triples about attribute triples cannot be converted by the transformation because the Property Graph model does not support metadata about (vertex) properties. Consequently, the second transformation is more limited in the RDF* data that it can handle than the the aforementioned lossless transformation.

Example 8. *The first line in the RDF* data in Example 3 represents a metadata triple about a relationship triple, and the fourth line represents represents a metadata triple about an attribute triple. Due to the existence of the latter metadata triple, the lossy transformation cannot be applied to the given sample data. However, suppose the sample data would not contain the last line. Then, Figure 4 illustrates the (simple) Property Graph that can be obtained by transforming this data based on the lossy transformation. A Cypher query for this Property Graph with the same intention as the queries in Examples 6 and 7 is given as follows.*

```

START a=node(*)
MATCH (a { foaf:name="Alice" })-[r:foaf:knows]->(p)
WHERE r.ex:certainty > 0.7
RETURN p.foaf:name

```

3.3 Third Transformation: Property Graphs to RDF*

The third transformation converts Property Graphs to RDF* data (which then may be transformed to standard RDF data [HT14]). The idea of the transformation is simple: Every edge (including its label) in a given Property Graph is represented as an ordinary RDF triple in the resulting RDF* data; the same holds for every vertex property. Any edge property is represented as a metadata triple whose subject is the triple representing the corresponding edge. The transformation gives users the freedom to choose patterns for generating IRIs that denote edge labels and properties keys, respectively. These IRIs become the predicates of triples in the resulting RDF* data.

Example 9. An RDF^{*} representation (given in Turtle^{*} format) that is the result of applying the proposed transformation to the Property Graph in Example 1 (cf. Figure 1) is given as follows:

```
@prefix p: <http://example.org/property/> .
@prefix r: <http://example.org/relationship/> .

_:b1 p:name "Stanley Kubrick" .
_:b1 p:birthyear 1928 .
_:b2 p:name "Orson Welles" .
_:b2 r:mentioned _:b1 .
<<_:b1 r:influencedBy _:b2>> p:certainty 0.9 .
```

The symbols `_:b1` and `_:b2` represent RDF blank nodes in Turtle and in Turtle^{*} format. Blank nodes can be understood as dataset-specific identifiers that cannot be referred to from other datasets (in contrast to IRIs). Hence, a blank node in some RDF or RDF^{*} data is similar to a vertex identity in a Property Graph. However, while this example assumes a mapping of vertices to blank nodes, the transformation also allows a user to specify a mapping of vertices to IRIs (cf. Definition 13).

For this transformation, there also exists a minor limitation: The transformation cannot be used for a Property Graph that contains distinct edges with the same start node, the same end node, and the same label.⁹ Section 7 elaborates more on this limitation and formalizes the transformation.

As a counterpart to the first two transformations, this third transformation enables RDF-based systems to import Property Graphs and execute SPARQL or SPARQL^{*} queries over the resulting RDF or RDF^{*} data. On the other hand, a Property Graph system may provide virtual RDF^{*} views of its Property Graphs that can be queried using SPARQL^{*}.

Example 10. Given the RDF^{*} representation in Example 9, a user can query the data originally represented in the Property Graph in Figure 1 by using SPARQL^{*} queries such as the following.

```
SELECT ?n WHERE {
  ?w p:name "Orson Welles" .
  <<?p r:influencedBy ?w>> p:certainty ?c .
  ?p p:name ?n .
}
ORDER BY ?c
```

The query returns an ordered list of persons influenced by Orson Welles; the order represents the certainty of whether Welles influenced these persons. An equivalent Cypher query that can be evaluated over the original Property Graph is given as follows.

```
START p=node(*)
MATCH (p)-[r:influencedBy]->(w { name="Orson Welles" })
RETURN p.name
ORDER BY r.certainty
```

The remainder of this document defines the proposed transformations formally. As a basis, Section 4 provides a formalization of the data models. Thereafter, Section 5 defines the lossless transformation of RDF^{*} data to RDF-like Property Graphs (as outlined in Section 3.1 above), Section 6 defines the lossy transformation of RDF^{*} data to simple Property Graphs (cf. Section 3.2 above), and Section 7 formalizes the transformation of Property Graphs to RDF^{*} (cf. Section 3.3 above).

⁹It remains to be seen whether this limitation has an impact in practice (there exist alternative, more explicit approaches to represent information in a Property Graph without having to use multiple edges between vertices).

4 Formalization of the Data Models

This section recalls the relevant definitions of RDF and RDF^{*}, provides a formal definition of the notion of a Property Graph, and identifies a class of RDF^{*} data that has an expressive power analogous to Property Graphs and, thus, can be used as a basis for the transformations in this document.

4.1 RDF^{*}

RDF^{*} is an extension of the RDF data model that makes metadata statements a first class citizen. This section provides the relevant parts of the formalization of RDF [CWL⁺14] and RDF^{*} [HT14].

Assume pairwise disjoint sets $\mathcal{I}$ (all IRIs), $\mathcal{B}$ (blank nodes), and $\mathcal{L}$ (literals). Each literal is associated with the IRI of a *datatype* [CWL⁺14]. Hence, formally, there exists a (total) mapping $\text{dtype} : \mathcal{L} \rightarrow \mathcal{I}$. Additionally, literals may be associated with a *language tag* [CWL⁺14]. Hence, there exists a partial mapping $\text{lang} : \mathcal{L} \rightarrow S_{\text{LTags}}$ such that S_{LTags} is the set of all strings that represent well-formed language tags as defined in [PD09].

An *RDF triple* is a tuple $\langle s, p, o \rangle \in (\mathcal{I} \cup \mathcal{B}) \times \mathcal{I} \times (\mathcal{I} \cup \mathcal{B} \cup \mathcal{L})$ and a set of RDF triples is called an *RDF graph*.

RDF^{*} extends such triples by permitting the embedding of a given triple in the subject or object position of another triple. Triples whose subject or object is an embedded triple represent some form of metadata. An embedded triple may itself be a metadata triple and, thus, may also contain embedded triples; and so forth. The following definition captures this notion.

Definition 1. Let $\tilde{\mathcal{T}}$ be an (infinite) set of tuples that is defined recursively as follows:

1. $\tilde{\mathcal{T}}$ includes all RDF triples, and
2. if $t \in \tilde{\mathcal{T}}$ and $t' \in \tilde{\mathcal{T}}$, then $\langle t, p, o \rangle \in \tilde{\mathcal{T}}$, $\langle s, p, t \rangle \in \tilde{\mathcal{T}}$, and $\langle t, p, t' \rangle \in \tilde{\mathcal{T}}$ for all $s \in (\mathcal{I} \cup \mathcal{B})$, $p \in \mathcal{I}$, and $o \in (\mathcal{I} \cup \mathcal{B} \cup \mathcal{L})$.

Moreover, for any tuple $\langle s, p, o \rangle \in \tilde{\mathcal{T}}$ and any natural number k , call the tuple *k-nested* if either:

1. $k = 0$ and the tuple $\langle s, p, o \rangle$ is an RDF triple, or
2. $k > 0$, for each (embedded) tuple $t' \in (\{s, o\} \cap \tilde{\mathcal{T}})$, there exists a k' such that $k' \leq k - 1$ and t' is k' -nested, and there exists an (embedded) tuple $t' \in (\{s, o\} \cap \tilde{\mathcal{T}})$ that is $(k - 1)$ -nested.

Then, an **RDF^{*} triple** is a tuple $t \in \tilde{\mathcal{T}}$ for which there exists a natural number k such that t is k -nested. A set of RDF^{*} triples is called an **RDF^{*} graph**.

Note that the notion of *k-nestedness* in Definition 1 rules out infinitely deep nesting of RDF^{*} triples.

Hereafter, the set of *all RDF^{*} triples* is denoted by $\mathcal{T}^*$, and, for any RDF^{*} triple $t \in \mathcal{T}^*$, $\text{Elmts}^+(t)$ denotes the set of all RDF terms and all RDF^{*} triples mentioned in t ; i.e., let $t = \langle s, p, o \rangle$, then $\text{Elmts}^+(t) = \{s, p, o\} \cup \{x' \in \text{Elmts}^+(x) \mid x \in \{s, o\} \cap \mathcal{T}^*\}$. An RDF^{*} triple t with $\text{Elmts}^+(t) \cap \mathcal{T}^* \neq \emptyset$ is called a *metadata triple* (note that any other RDF^{*} triple is an ordinary RDF triple).

Overloading function Elmts^+ , for any RDF^{*} graph G^* , $\text{Elmts}^+(G^*) = \bigcup_{t \in G^*} \text{Elmts}^+(t)$. Furthermore, $\text{Emb}^+(G^*)$ denotes the set of all RDF^{*} triples that are (recursively) embedded in RDF^{*} triples of RDF^{*} graph G^* ; i.e., $\text{Emb}^+(G^*) = \text{Elmts}^+(G^*) \cap \mathcal{T}^*$.

Example 11. Formally, the Turtle^{*} document in Example 3 represents the following RDF^{*} graph.

$$G_{\text{ex}}^* = \{ \langle \langle \text{ex:alice, foaf:knows, ex:bob} \rangle, \text{ex:certainty, 0.5} \rangle, \\ \langle \text{ex:alice, foaf:name, "Alice"} \rangle, \\ \langle \text{ex:bob, foaf:name, "Bob"} \rangle, \\ \langle \langle \text{ex:bob, foaf:age, 23} \rangle, \text{ex:certainty, 0.9} \rangle \}$$

Note that this RDF^{*} graph consists of four RDF^{*} triples, two of which are ordinary RDF triples and the other are metadata triples. It holds that

$$\text{Elmts}^+(G_{\text{ex}}^*) = \{ \text{ex:alice, foaf:knows, ex:bob}, \langle \text{ex:alice, foaf:knows, ex:bob} \rangle, \text{ex:certainty, 0.5}, \\ \text{foaf:name, "Alice", "Bob", foaf:age, 23}, \langle \text{ex:bob, foaf:age, 23} \rangle, 0.9 \}$$

and, thus,

$$\text{Emb}^+(G_{\text{ex}}^*) = \{ \langle \text{ex:alice, foaf:knows, ex:bob} \rangle, \langle \text{ex:bob, foaf:age, 23} \rangle \}.$$

For a more detailed discussion of RDF^{*}, including a well-defined transformation from RDF^{*} graphs to ordinary RDF graphs, an RDF^{*}-enabled extension of the Turtle syntax (as used in Examples 3 and 9), and a definition of SPARQL^{*} (the RDF^{*}-aware extension of SPARQL; cf. Examples 7 and 10), refer to [HT14].

4.2 Property Graphs

This section defines the notion of a Property Graph formally. As a basis for this definition, assume a (programming language specific) set $\mathfrak{D}$ of data types, including a type for strings—hereafter, denoted by $\mathcal{S}$ (i.e., $\mathcal{S} \in \mathfrak{D}$). Note that $\mathfrak{D}$ may also contain data types for collections. For each data type $\mathcal{D} \in \mathfrak{D}$, let $\text{dom}(\mathcal{D})$ denote the value space of $\mathcal{D}$; i.e., the set of all possible values of type $\mathcal{D}$. Hence, $\text{dom}(\mathcal{S})$ is the set of all strings. Furthermore, assume that values are unique; that is, for any value $x \in \text{dom}(\mathcal{D})$ of any data type $\mathcal{D} \in \mathfrak{D}$, there does not exist another data type $\mathcal{D}' \in \mathfrak{D} \setminus \{\mathcal{D}\}$ such that $x \in \text{dom}(\mathcal{D}')$.

Given such a set of data types $\mathfrak{D}$, the notion of a *Property Graph* can be formalized as follows.

Definition 2. Let P be the (infinite) set of all possible properties; that is, a pair $p = \langle k, v \rangle$ where $k \in \text{dom}(\mathcal{S})$ and $v \in \bigcup_{\mathcal{D} \in \mathfrak{D}} \text{dom}(\mathcal{D})$; i.e., $P = \text{dom}(\mathcal{S}) \times \bigcup_{\mathcal{D} \in \mathfrak{D}} \text{dom}(\mathcal{D})$. A **Property Graph** is a tuple $G = \langle V, E, \text{src}, \text{tgt}, \text{lbl}, \mathfrak{P} \rangle$ such that

- $\langle V, E, \text{src}, \text{tgt}, \text{lbl} \rangle$ is an edge-labeled directed multigraph with
 - a set of vertices V ,
 - a set of edges E ,
 - a function $\text{src}: E \rightarrow V$ that associates each edge with its source vertex,
 - a function $\text{tgt}: E \rightarrow V$ that associates each edge with its target vertex, and
 - a function $\text{lbl}: E \rightarrow \text{dom}(\mathcal{S})$ that associates each edge its label; and
 - $\mathfrak{P}$ is a function $\mathfrak{P}: (V \cup E) \rightarrow 2^P$.
-

The following definition identifies a more restricted class of Property Graphs; that is, Property Graphs in which, for each vertex or edge, all properties of this vertex or edge have a unique key.

Definition 3. A Property Graph $G = \langle V, E, src, tgt, lbl, \mathfrak{P} \rangle$ is **property-unique** if, for each vertex or edge $x \in (V \cup E)$, $k_1 \neq k_2$ for all pairs of distinct properties $\langle k_1, v_1 \rangle, \langle k_2, v_2 \rangle \in \mathfrak{P}(x)$.

Example 12. Reconsider the Property Graph introduced in Example 1 (cf. Figure 1). A formal representation of this graph is given by the tuple $G_{\text{ex}} = \langle V_{\text{ex}}, E_{\text{ex}}, src_{\text{ex}}, tgt_{\text{ex}}, lbl_{\text{ex}}, \mathfrak{P}_{\text{ex}} \rangle$ consisting of the following elements:

- $V_{\text{ex}} = \{\text{Kubrick}, \text{Welles}\}$
- $E_{\text{ex}} = \{e_1, e_2\}$
- $src_{\text{ex}}(e_1) = \text{Welles}, tgt_{\text{ex}}(e_1) = \text{Kubrick}, lbl_{\text{ex}}(e_1) = \text{"mentioned"}$
- $src_{\text{ex}}(e_2) = \text{Kubrick}, tgt_{\text{ex}}(e_2) = \text{Welles}, lbl_{\text{ex}}(e_2) = \text{"influencedBy"}$
- $\mathfrak{P}_{\text{ex}}(\text{Kubrick}) = \{ \langle \text{"name"}, \text{"Stanley Kubrick"} \rangle, \langle \text{"birthyear"}, 1928 \rangle \}$
- $\mathfrak{P}_{\text{ex}}(\text{Welles}) = \{ \langle \text{"name"}, \text{"Orson Welles"} \rangle \}$
- $\mathfrak{P}_{\text{ex}}(e_1) = \emptyset$
- $\mathfrak{P}_{\text{ex}}(e_2) = \{ \langle \text{"certainty"}, 0.8 \rangle \}$

Apparently, this example Property Graph G_{ex} is property-unique.

While the given definition of a Property Graph does not restrict the types of property values, the remainder of this document assumes that all values of the data types in $\mathfrak{D}$ can be mapped to distinct RDF literals. More precisely, the following definitions assume a bijective function $vm : \mathcal{V} \rightarrow \mathcal{L}^*$ such that $\mathcal{V} = \bigcup_{\mathcal{D} \in \mathfrak{D}} \text{dom}(\mathcal{D})$ and $\mathcal{L}^*$ is a set of literals; i.e., $\mathcal{L}^* \subseteq \mathcal{L}$ and $|\mathcal{L}^*| = |\mathcal{V}|$. Hereafter, function vm is called the *value-to-literal mapping*. Note that this assumption rules out Property Graphs with property values that are of some data type for collections.

4.3 Property Graph Convertibility of RDF^{*} graphs

Before going into the details of the transformations it is important to note that the RDF^{*} data model is more expressive than the Property Graphs model. For instance, RDF^{*} allows for an arbitrarily deep nesting of metadata triples, whereas a Property Graph cannot contain additional metadata about a property of a vertex or an edge (i.e., a property in a Property Graph cannot be annotated with properties itself). As a consequence, any transformation from RDF^{*} graphs to Property Graphs that adapts the natural approach of representing metadata triples as edge properties is possible only for specific RDF^{*} graphs. Informally, these RDF^{*} graphs must satisfy the following conditions:

1. Metadata triples are not nested within one another.
2. Metadata triples embed triples as their subject only (not as their object).
3. The object of any metadata triple must be a literal.

4. For any literal in the triples it must be possible to convert the literal to a data value.

The following definition formalizes these conditions.

Definition 4. An RDF^{*} graph G^* is **Property Graph convertible (PG-convertible)** for short) if each RDF^{*} triple $t \in G^*$ with $t = \langle s, p, o \rangle$ has the following properties:

1. If $s \in \mathcal{T}^*$, then $\text{Elmts}^+(s) \cap \mathcal{T}^* = \emptyset$. (i.e., s is not a metadata triple; cf. condition 1 above)
 2. It holds that $o \notin \mathcal{T}^*$. (i.e., o is not an embedded triple; cf. condition 2 above)
 3. If $s \in \mathcal{T}^*$, then $o \in \mathcal{L}$. (i.e., o is a literal if t is a metadata triple; cf. condition 3 above)
 4. For every literal $l \in \mathcal{L}$, if $l \in \text{Elmts}^+(t)$, then $l \in \text{dom}(vm^{-1})$; where vm^{-1} is the inverse of the aforementioned (bijective) value-to-literal mapping vm . (cf. condition 4 above)
-

Example 13. The sample RDF^{*} graph G_{ex}^* (cf. Example 11) is PG-convertible.

An even stronger notion of PG-convertibility is necessary for the second transformation. As mentioned in Section 3.2, this transformation can be applied only to RDF^{*} graphs that satisfy the conditions for PG-convertibility and, additionally, do not contain metadata triples about attribute triples. The following definition captures this stronger notion of PG-convertibility.

Definition 5. An RDF^{*} graph G^* is **strongly PG-convertible** if G^* is PG-convertible and, for every (embedded) RDF^{*} triple $\langle s, p, o \rangle \in \text{Emb}^+(G^*)$, it holds that $o \notin \mathcal{L}$.

Example 14. The sample RDF^{*} graph G_{ex}^* (cf. Example 11) is not strongly PG-convertible because it contains the metadata triple $\langle \langle \text{ex:} \text{bob}, \text{foaf:age}, 23 \rangle, \text{ex:certainty}, 0.9 \rangle$. However, the following subgraph $G_{\text{ex}'}^* \subseteq G_{\text{ex}}^*$, which does not contain this metadata triple, is strongly PG-convertible.

$$G_{\text{ex}'}^* = \{ \langle \langle \text{ex:alice}, \text{foaf:knows}, \text{ex:bob} \rangle, \text{ex:certainty}, 0.5 \rangle, \\ \langle \text{ex:alice}, \text{foaf:name}, \text{"Alice"} \rangle, \\ \langle \text{ex:bob}, \text{foaf:name}, \text{"Bob"} \rangle \}.$$

5 Transforming RDF^{*} Graphs to RDF-like Property Graphs

This section formalizes the lossless transformation of RDF^{*} graphs to Property Graphs as outlined in Section 3.1. Recall that this transformation represents any ordinary RDF triple (which is not a metadata triple) as an edge, and metadata triples are represented as edge properties. Due to the aforementioned differences in the expressiveness of RDF^{*} and Property Graphs, the transformation can be applied only to PG-convertible RDF^{*} graphs (cf. Definition 4).

Another requirement for the transformation is an unambiguous mapping of any possible IRI to a distinct string. The usual string representation of IRIs is sufficient for this purpose. Hence, the following definition assumes an injective function $im: \mathcal{I} \rightarrow \text{dom}(\mathcal{S})$. Hereafter, this function im is called the *IRI-to-string mapping*.

Given these preliminaries, the transformation is defined as follows.

Definition 6. Let G^* be an RDF^{*} graph that is PG-convertible. Furthermore, let:

$$\begin{aligned} \text{meta}(G^*) &= \{t \in G^* \mid \text{Elmts}^+(t) \cap \mathcal{T}^* \neq \emptyset\}, \\ \text{ord}(G^*) &= (G^* \cup \text{Emb}^+(G^*)) \setminus \text{meta}(G^*), \text{ and} \\ \text{SOTerms}^+(G^*) &= \{x \in (\mathcal{I} \cup \mathcal{B} \cup \mathcal{L}) \mid \langle s, p, o \rangle \in \text{ord}(G^*) \text{ and } x \in \{s, o\}\}; \end{aligned}$$

i.e., $\text{meta}(G^*)$ is the set of all metadata triples in G^* , $\text{ord}(G^*)$ is the set of all ordinary RDF triples in G^* , and $\text{SOTerms}^+(G^*)$ is the set of all RDF terms in the subject position or object position of some ordinary triple in G^* . The **RDF-like Property Graph representation** of G^* is the Property Graph $G = \langle V, E, \text{src}, \text{tgt}, \text{lbl}, \mathfrak{P} \rangle$ that has the following properties:

1. The set of vertices V contains $n_V = |\text{SOTerms}^+(G^*)|$ vertices, each of which represents a different RDF term in $\text{SOTerms}^+(G^*)$; i.e., there exists a bijective function $v: \text{SOTerms}^+(G^*) \rightarrow V$ that maps each RDF term $x \in \text{SOTerms}^+(G^*)$ to a (different) vertex $v(x) \in V$.
2. For each IRI $u \in \mathcal{I}$ with $u \in \text{SOTerms}^+(G^*)$, the set of properties $\mathfrak{P}(v(u))$ of vertex $v(u) \in V$ is defined as $\mathfrak{P}(v(u)) = \{\langle \text{"kind"}, \text{"IRI"} \rangle, \langle \text{"IRI"}, \text{im}(u) \rangle\}$ where im is the aforementioned IRI-to-string mapping.
3. For each blank node $b \in \mathcal{B}$ with $b \in \text{SOTerms}^+(G^*)$, the set of properties $\mathfrak{P}(v(b))$ of vertex $v(b) \in V$ is defined as $\mathfrak{P}(v(b)) = \{\langle \text{"kind"}, \text{"blank node"} \rangle\}$.
4. For each literal $l \in \mathcal{L}$ with $l \in \text{SOTerms}^+(G^*)$, the set of properties $\mathfrak{P}(v(l))$ of vertex $v(l) \in V$ is defined as

$$\mathfrak{P}(v(l)) = \{\langle \text{"kind"}, \text{"literal"} \rangle, \langle \text{"literal"}, \text{vm}^{-1}(l) \rangle, \langle \text{"datatype"}, \text{im}(\text{dtype}(l)) \rangle\} \cup \text{lang}$$

where vm^{-1} is the inverse of the aforementioned (bijective) value-to-literal mapping vm , and

$$\text{lang} = \begin{cases} \{\langle \text{"language"}, \text{lang}(l) \rangle\} & \text{if } l \in \text{dom}(\text{lang}), \\ \emptyset & \text{else.} \end{cases}$$

5. The set of edges E contains $n_E = |\text{ord}(G^*)|$ edges, each of which represents a different RDF triple $t \in \text{ord}(G^*)$. Hence, there exists a bijective function $e: \text{ord}(G^*) \rightarrow E$ that maps each $t \in \text{ord}(G^*)$ to a (different) edge $e(t) \in E$.
6. For each triple $t \in \text{ord}(G^*)$ with $t = \langle s, p, o \rangle$, the label of edge $e(t) \in E$ is $\text{im}(p)$, and the source and target vertex of $e(t)$ is $v(s)$ and $v(o)$, respectively; i.e., $\text{lbl}(e(t)) = \text{im}(p)$, $\text{src}(e(t)) = v(s)$, and $\text{tgt}(e(t)) = v(o)$.
7. For each triple $t \in \text{ord}(G^*)$, the set of properties $\mathfrak{P}(e(t))$ of edge $e(t) \in E$ is defined as

$$\mathfrak{P}(e(t)) = \bigcup_{\langle s, p, o \rangle \in \text{ord}(G^*) \text{ such that } s=t} \{\langle \text{im}(p), \text{vm}^{-1}(o) \rangle\}.$$

Example 15. Since the sample RDF^{*} graph G_{ex}^* (cf. Example 11) is PG-convertible, the given transformation can be used to convert G_{ex}^* to a Property Graph. Figure 3 (in Example 4) illustrates the resulting Property Graph G'_{ex} , which is the RDF-like Property Graph representation of G_{ex}^* . Formally, this Property Graph is given by the tuple $G'_{\text{ex}} = \langle V'_{\text{ex}}, E'_{\text{ex}}, \text{src}'_{\text{ex}}, \text{tgt}'_{\text{ex}}, \text{lbl}'_{\text{ex}}, \mathfrak{P}'_{\text{ex}} \rangle$ that consists of the following elements:

- $V'_{\text{ex}} = \{v_1, v_2, v_3, v_4, v_5\}$
- $E'_{\text{ex}} = \{e_1, e_2, e_4, e_5\}$
- $\text{src}'_{\text{ex}}(e_1) = v_1$, $\text{tgt}'_{\text{ex}}(e_1) = v_3$, and $\text{lbl}'_{\text{ex}}(e_1) = \text{"http://xmlns.com/foaf/0.1/name"}$
- $\text{src}'_{\text{ex}}(e_2) = v_1$, $\text{tgt}'_{\text{ex}}(e_2) = v_2$, and $\text{lbl}'_{\text{ex}}(e_2) = \text{"http://xmlns.com/foaf/0.1/knows"}$
- $\text{src}'_{\text{ex}}(e_3) = v_2$, $\text{tgt}'_{\text{ex}}(e_3) = v_4$, and $\text{lbl}'_{\text{ex}}(e_3) = \text{"http://xmlns.com/foaf/0.1/name"}$
- $\text{src}'_{\text{ex}}(e_4) = v_2$, $\text{tgt}'_{\text{ex}}(e_4) = v_5$, and $\text{lbl}'_{\text{ex}}(e_4) = \text{"http://xmlns.com/foaf/0.1/age"}$
- $\mathfrak{P}'_{\text{ex}}(v_1) = \{ \langle \text{"kind"}, \text{"IRI"} \rangle, \langle \text{"IRI"}, \text{"http://example.org/alice"} \rangle \}$
- $\mathfrak{P}'_{\text{ex}}(v_2) = \{ \langle \text{"kind"}, \text{"IRI"} \rangle, \langle \text{"IRI"}, \text{"http://example.org/bob"} \rangle \}$
- $\mathfrak{P}'_{\text{ex}}(v_3) = \{ \langle \text{"kind"}, \text{"literal"} \rangle, \langle \text{"literal"}, \text{"Alice"} \rangle \}$
- $\mathfrak{P}'_{\text{ex}}(v_4) = \{ \langle \text{"kind"}, \text{"literal"} \rangle, \langle \text{"literal"}, \text{"Bob"} \rangle \}$
- $\mathfrak{P}'_{\text{ex}}(v_5) = \{ \langle \text{"kind"}, \text{"literal"} \rangle, \langle \text{"literal"}, 23 \rangle \}$
- $\mathfrak{P}'_{\text{ex}}(e_1) = \emptyset$
- $\mathfrak{P}'_{\text{ex}}(e_2) = \{ \langle \text{"http://example.org/certainty"}, 0.5 \rangle \}$
- $\mathfrak{P}'_{\text{ex}}(e_3) = \emptyset$
- $\mathfrak{P}'_{\text{ex}}(e_4) = \{ \langle \text{"http://example.org/certainty"}, 0.9 \rangle \}$

Remark 1. For any PG-convertible RDF^{*} graph G^* it holds that if G^* contains two (distinct) metadata triples that differ only in their objects (i.e., there exist $\langle s, p, o \rangle, \langle s', p', o' \rangle \in G^*$ such that $s = s'$, $p = p'$, $o \neq o'$, $s \in \mathcal{T}^*$, and $s' \in \mathcal{T}^*$), then the RDF-like Property Graph representation of G^* is *not* property-unique.

While the given transformation is lossless (i.e., resulting Property Graphs contain all information present in the original RDF^{*} graph), some use cases may have the stronger requirement that the original RDF^{*} graph can be reconstructed *exactly* from its RDF-like Property Graph representation. Hence, for these use cases the transformation must be invertible.

To ensure an invertible transformation, any RDF^{*} graph to be transformed must not contain redundant RDF^{*} triples; that is, RDF^{*} triples that are embedded in metadata triples in the RDF^{*} graph and, additionally, appear directly (as a separate element) in the RDF^{*} graph. The following definition captures this notion of redundancy formally.

Definition 7. Let G^* be an RDF^{*} graph and let $t \in G^*$ be an RDF^{*} triple in G^* . RDF^{*} triple t is **redundant** in G^* if there exists another (metadata) triple $t' \in G^*$ such that $t \in \text{Elmts}^+(t')$.

RDF^{*} graphs must have the following minimality property to ensure an invertible transformation.

Definition 8. An RDF^{*} graph G^* is **minimal** if there does not exist an RDF^{*} triple $t \in G^*$ that is redundant in G^* .

Example 16. *It is easily verified that the sample RDF^{*} graph G_{ex}^* (cf. Example 11) is minimal.*

Remark 2. Note that every RDF^{*} graph can be converted into a minimal RDF^{*} graph by removing all redundant RDF^{*} triples. Then, any unfolded RDF graph [HT14] of the resulting (minimal) RDF^{*} graph is equivalent to the corresponding unfolded RDF graph of the original RDF^{*} graph.

6 Transforming RDF^{*} Graphs to Simple Property Graphs

This section formalizes the transformation of RDF^{*} graphs to Property Graphs as outlined in Section 3.2. Recall that this transformation represents relationship triples (whose object is an IRI or a blank node) as edges; attribute triples (whose object is a literal) are converted into vertex properties for their subject. Metadata triples about relationship triples are converted into edge properties, and metadata triples about attribute triples cannot be converted by the transformation because the Property Graph model does not support metadata about (vertex) properties. The following definition captures this approach, which is applicable only to RDF^{*} graphs that are strongly PG-convertible (cf. Definition 5).

Definition 9. Let G^* be an RDF^{*} graph that is strongly PG-convertible. Furthermore, let:

$$\begin{aligned} \text{meta}(G^*) &= \{t \in G^* \mid \text{Elmts}^+(t) \cap \mathcal{T}^* \neq \emptyset\}, \\ \text{ord}(G^*) &= (G^* \cup \text{Emb}^+(G^*)) \setminus \text{meta}(G^*), \\ \text{ordA}(G^*) &= \{\langle s, p, o \rangle \in \text{ord}(G^*) \mid o \in \mathcal{L}\}, \\ \text{ordR}(G^*) &= \{\langle s, p, o \rangle \in \text{ord}(G^*) \mid o \in (\mathcal{I} \cup \mathcal{B})\}, \text{ and} \\ \text{SO}^+(G^*) &= \{x \in (\mathcal{I} \cup \mathcal{B}) \mid \langle s, p, o \rangle \in \text{ord}(G^*) \text{ and } x \in \{s, o\}\}; \end{aligned}$$

i.e., $\text{meta}(G^*)$ is the set of all metadata triples in G^* , $\text{ord}(G^*)$ is the set of all ordinary RDF triples in G^* , $\text{ordA}(G^*)$ is the set of all ordinary attribute triples in G^* , $\text{ordR}(G^*)$ is the set of all ordinary relationship triples in G^* , and $\text{SO}^+(G^*)$ is the set of all IRIs and blank nodes in the subject position or object position of some ordinary triple in G^* . The **simple Property Graph representation** of G^* is the Property Graph $G = \langle V, E, \text{src}, \text{tgt}, \text{lbl}, \mathfrak{P} \rangle$ that has the following properties:

1. The set of vertices V contains $n_V = |\text{SO}^+(G^*)|$ vertices, each of which represents a different IRI or blank node $x \in \text{SO}^+(G^*)$. Hence, there exists a bijective function $v : \text{SO}^+(G^*) \rightarrow V$ that maps each $x \in \text{SO}^+(G^*)$ to a (different) vertex $v(x) \in V$.
2. For each IRI $u \in \mathcal{I}$ with $u \in \text{SO}^+(G^*)$, the set of properties $\mathfrak{P}(v(u))$ of vertex $v(u) \in V$ is defined as

$$\mathfrak{P}(v(u)) = \{\langle \text{"IRI"}, \text{im}(u) \rangle\} \cup \bigcup_{\langle s, p, o \rangle \in \text{ordA}(G^*) \text{ such that } s=u} \{\langle \text{im}(p), \text{vm}^{-1}(o) \rangle\},$$

where im is the aforementioned IRI-to-string mapping, and vm^{-1} is the inverse of the aforementioned (bijective) value-to-literal mapping vm .

3. For each blank node $b \in \mathcal{B}$ with $b \in \text{SO}^+(G^*)$, the set of properties $\mathfrak{P}(v(b))$ of vertex $v(b) \in V$ is defined as

$$\mathfrak{P}(v(b)) = \bigcup_{\langle s,p,o \rangle \in \text{ordR}(G^*) \text{ such that } s=b} \{ \langle \text{im}(p), \text{vm}^{-1}(o) \rangle \}.$$

4. The set of edges E contains $n_E = |\text{ordR}(G^*)|$ edges, each of which represents a different (relationship) triple $t \in \text{ordR}(G^*)$. Hence, there exists a bijective function $e: \text{ordR}(G^*) \rightarrow E$ that maps each $t \in \text{ordR}(G^*)$ to a (different) edge $e(t) \in E$.
5. For each (relationship) triple $t \in \text{ordR}(G^*)$ with $t = \langle s, p, o \rangle$, the label of edge $e(t) \in E$ is $\text{im}(p)$, and the source and target vertex is $v(s)$ and $v(o)$, respectively; i.e., $\text{lbl}(e(t)) = \text{im}(p)$, $\text{src}(e(t)) = v(s)$, and $\text{tgt}(e(t)) = v(o)$.
6. For each (relationship) triple $t \in \text{ordR}(G^*)$, the set of properties $\mathfrak{P}(e(t))$ of edge $e(t) \in E$ is defined as

$$\mathfrak{P}(e(t)) = \bigcup_{\langle s,p,o \rangle \in \text{meta}(G^*) \text{ such that } s=t} \{ \langle \text{im}(p), \text{vm}^{-1}(o) \rangle \}.$$

Example 17. The RDF^{*} graph G_{ex}^* , as introduced in Example 14, is strongly PG-convertible and, thus, it can be converted to a simple Property Graph representation. Figure 4 (in Example 8) illustrates the resulting Property Graph G_{ex}'' , which is the simple Property Graph representation of G_{ex}^* . Formally, this Property Graph is given by the tuple $G_{\text{ex}}'' = \langle V_{\text{ex}}'', E_{\text{ex}}'', \text{src}_{\text{ex}}'', \text{tgt}_{\text{ex}}'', \text{lbl}_{\text{ex}}'', \mathfrak{P}_{\text{ex}}'' \rangle$ that consists of the following elements:

- $V_{\text{ex}}'' = \{v_1, v_2\}$
- $E_{\text{ex}}'' = \{e_1\}$
- $\text{src}_{\text{ex}}''(e_1) = v_1$, $\text{tgt}_{\text{ex}}''(e_1) = v_2$, and $\text{lbl}_{\text{ex}}''(e_1) = \text{"http://xmlns.com/foaf/0.1/knows"}$
- $\mathfrak{P}_{\text{ex}}''(v_1) = \{ \langle \text{"IRI"}, \text{"http://example.org/alice"} \rangle, \langle \text{"http://xmlns.com/foaf/0.1/name"}, \text{"Alice"} \rangle \}$
- $\mathfrak{P}_{\text{ex}}''(v_2) = \{ \langle \text{"IRI"}, \text{"http://example.org/bob"} \rangle, \langle \text{"http://xmlns.com/foaf/0.1/name"}, \text{"Bob"} \rangle \}$
- $\mathfrak{P}_{\text{ex}}''(e_1) = \{ \langle \text{"http://example.org/certainty"}, 0.5 \rangle \}$

Remark 3. For any strongly PG-convertible RDF^{*} graph G^* it holds that if G^* contains two (distinct) RDF^{*} triples that differ only in their objects and these objects are literals (i.e., there exist $\langle s, p, o \rangle, \langle s', p', o' \rangle \in G^* \cup \text{Emb}^+(G^*)$ such that $s = s'$, $p = p'$, $o \neq o'$, $o \in \mathcal{L}$, and $o' \in \mathcal{L}$), then the simple Property Graph representation of G^* is *not* property-unique.

7 Transforming Property Graphs to RDF^{*} Graphs

This section defines the transformation of Property Graphs to RDF^{*} graphs as described in Section 3.3. The idea of this transformation is to represent each vertex of a given Property Graph either by a blank node or an IRI; each edge is represented as an ordinary (non-metadata) triple whose subject and object are the blank node or IRI of the adjacent vertices and whose predicate is a IRI that denotes the label of the edge; moreover, vertex properties are also represented as

ordinary triples, and edge properties are represented as metadata triples whose subject is the triple for the corresponding edge.

While representing each edge (including its label) as a single triple is perhaps the most intuitive approach to transform such edges, this approach has the following shortcoming: If there are two (or more) distinct edges that connect the same vertices and have the same label (but may have different properties), the approach would represent both edges by a single triple. As a result, this triple would not represent any one of the edges unambiguously when embedded in a metadata triple for a property of the edge. To avoid this problem, the transformation is restricted to Property Graphs that do *not* contain distinct edges with the same source vertex, the same target vertex, and the same label. Hereafter, these Property Graphs are called *edge-unique*.

Definition 10. A Property Graph $G = \langle V, E, src, tgt, lbl, \mathfrak{P} \rangle$ is **edge-unique** if there does not exist a pair of edges $(e, e') \in E \times E$ such that $e \neq e'$, $src(e) = src(e')$, $tgt(e) = tgt(e')$, and $lbl(e) = lbl(e')$.

Example 18. The Property Graph G_{ex} in Example 12 is edge-unique (cf. Figure 1).

Remark 4. Note that the requirement of edge-uniqueness does not present a restriction on the expressiveness of Property Graphs. Information represented by introducing multiple edges between vertices can also be modeled by using an alternative, more explicit approach. For instance, the relationship captured by each of the multiple edges may be modeled as a separate vertex.

The transformation assumes three user-specified templates for generating IRIs. The first two of these templates can be used to generate IRIs that denote arbitrary edge labels and arbitrary properties keys, respectively. The following two mappings capture the notion of these templates formally.

Definition 11. An **edge label mapping** lm is a bijective function $lm: \text{dom}(\mathcal{S}) \rightarrow I$ such that I is a set of IRIs; i.e., $I \subseteq \mathcal{I}$ and $|I| = |\text{dom}(\mathcal{S})|$. (Recall that $\text{dom}(\mathcal{S})$ is the set of all strings.)

Definition 12. A **property key mapping** km is a bijective function $km: \text{dom}(\mathcal{S}) \rightarrow I$ such that I is a set of IRIs; i.e., $I \subseteq \mathcal{I}$ and $|I| = |\text{dom}(\mathcal{S})|$. (Recall that $\text{dom}(\mathcal{S})$ is the set of all strings.)

Example 19. An example of a property key mapping is a function km_{ex} such that, for every string $s \in \text{dom}(\mathcal{S})$, function km_{ex} returns the IRI `http://example.org/property/urlenc(s)` where `urlenc(s)` is the URL encoded version of string s . Similarly, lm_{ex} is an edge label mapping such that, for every string $s \in \text{dom}(\mathcal{S})$, lm_{ex} returns the IRI `http://example.org/relationship/urlenc(s)`.

Remark 5. While a user-specified edge label mapping (resp. a property key mapping) can be used for transforming different Property Graphs, this should be done only if the same labels (resp. the same property keys) in these different Property Graphs have the same meaning. If that is not the case, a different edge label mapping (resp. property key mapping) should be used.

The third user-specified template can be used to generate IRIs or blank nodes for the vertices of a Property Graph. Formally, the following mapping captures such a template.

Definition 13. Let $G = \langle V, E, src, tgt, lbl, \mathfrak{P} \rangle$ be a Property Graph. A **vertex identity mapping** id for G is an injective function $id: V \rightarrow (\mathcal{B} \cup \mathcal{I})$.

The transformation can now be formalized as follows.

Definition 14. Let $G = \langle V, E, src, tgt, lbl, \mathfrak{P} \rangle$ be a Property Graph that is property-unique (cf. Definition 3) and edge-unique, let id be a vertex identity mapping for G , let lm be an edge label mapping, and, for each edge $e \in E$, let t_e be the RDF triple $\langle id(src(e)), lm(lbl(e)), id(tgt(e)) \rangle$. Then, given a property key mapping km (and the value-to-literal mapping vm), the (id, lm, km) -specific **RDF^{*} representation** of G is the set of RDF^{*} triples $G^* = G_{vp}^* \cup G_{ep}^* \cup G_{en}^*$ that consists of the following three subsets:

$$G_{vp}^* = \{ \langle id(v), km(k), vm(x) \rangle \mid v \in V \text{ and } \langle k, x \rangle \in \mathfrak{P}(v) \} \quad (\text{vertex properties})$$

$$G_{ep}^* = \{ \langle t_e, km(k), vm(x) \rangle \mid e \in E \text{ and } \langle k, x \rangle \in \mathfrak{P}(e) \} \quad (\text{edges with properties})$$

$$G_{en}^* = \{ t_e \mid e \in E \text{ and } \mathfrak{P}(e) = \emptyset \} \quad (\text{edges without properties})$$

Example 20. Recall the sample Property Graph G_{ex} as given in Example 12 (and illustrated in Figure 1). Assume a vertex identity mapping id_{ex} that maps each vertex in G_{ex} to a distinct blank node; i.e., $id_{ex}(\text{Kubrick}) = b_K$ and $id_{ex}(\text{Welles}) = b_W$, with $b_K, b_W \in \mathcal{B}$. Then, the $(id_{ex}, lm_{ex}, km_{ex})$ -specific RDF^{*} representation of Property Graph G_{ex} is the following RDF^{*} graph (Example 9 provides a Turtle^{*} serialization of this RDF^{*} graph):

$$\{ \langle b_K, \text{http://example.org/property/name}, "Stanley Kubrick" \rangle, \\ \langle b_K, \text{http://example.org/property/birthyear}, 1928 \rangle, \\ \langle b_W, \text{http://example.org/property/name}, "Orson Welles" \rangle, \\ \langle b_W, \text{http://example.org/relationship/mentioned}, b_K \rangle, \\ \langle \langle b_K, \text{http://example.org/relationship/influencedBy}, b_W \rangle, \\ \text{http://example.org/property/certainty}, 0.8 \rangle \}.$$

8 Acknowledgements

The author would like to thank a number of people for providing feedback on the proposal, as well as on earlier versions of this document. In alphabetical order, these are: Bryan Thompson, Cosmin Basca, Grant Weddell, Juan Sequeda, Kavitha Srinivas, Maria-Esther Vidal, Mike Personick, Peter Boncz, and Tamer Özsu.

References

- [CWL⁺14] Richard Cyganiak, David Wood, Markus Lanthaler, Graham Klyne, Jeremy J. Carroll, and Brian McBride. RDF 1.1 Concepts and Abstract Syntax. W3C Recommendation, Online at <http://www.w3.org/TR/rdf11-concepts/>, February 2014.
- [HPS14] Patrick J. Hayes and Peter F. Patel-Schneider. RDF 1.1 Semantics. W3C Recommendation, Online at <http://www.w3.org/TR/rdf11-mt/>, February 2014.
- [HT14] Olaf Hartig and Bryan Thompson. Foundations of an Alternative Approach to Reification in RDF. *CoRR*, abs/1406.3399, 2014. Online: <http://arxiv.org/abs/1406.3399>.

- [PC14] Eric Prud'hommeaux and Gavin Carothers. RDF 1.1 Turtle. W3C Recommendation, Online at <http://www.w3.org/TR/turtle/>, February 2014.
- [PD09] A. Phillips and M. Davis. Tags for Identifying Languages. IETF Best Current Practice, Online at <http://tools.ietf.org/html/bcp47>, September 2009.
- [RWE13] Ian Robinson, Jim Webber, and Emil Eifré. *Graph Databases*. O'Reilly Media, 2013.